pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Branches and Pull Requests

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: branches and pull requests

Today

1. Why branches: main is what is deployed
2. A branch of your own, read with diff_table
3. Two kinds of merge: fast-forward, and after a colleague
4. A conflict on purpose: keep both, the tests settle it
5. A pull request on GitHub, recorded and masked
6. Review by diff, before anything merges
7. Three ways to merge: the course squashes
8. Deleting merged branches, here and on the remote

Three functions, three branches

`diff_table`

what a change adds and removes

`branch_table`

ahead and behind main

`conflict_files`

what a merge left unmerged

No notebook cell reaches GitHub: the pull request is shown from its masked transcripts.

How a branch works

The problem	Work has to happen without touching main, which is what is deployed
The method	A commit points at its parent; a branch is a name that points at one commit
A new commit	The branch's name moves forward; every other name stays where it was
You control	What the branch starts from, and when it is merged back
What comes out	A line of commits beside main; git branch and status -sb say where each name points

Why branches

main is what is deployed

From day 3 GitHub tests it on every change; from day 7 Vercel serves what it holds.

branch

work beside main

→

review

tested, read by another

→

merge

into main

→

delete

its job is done

A branch is a name pointing at a commit. It costs nothing.

A branch of your own

```
git switch -c diff-table
# edit, test
git commit -m "..."
git push -u origin diff-table
```

```
branch 'diff-table' set up to track 'origin/diff-table'.
To remote.git
 * [new branch]      diff-table -> diff-table
```

`diff_table(work, "main", "diff-table")`

```
            file  added  removed
     deploykit.py     18        0
test_deploykit.py     26        0
lines added: 44; removed: 0
```

A function and its test, nothing else touched: what a reviewer reads first.

How a merge works

The problem	Two lines of commits have to become one
Fast-forward	main has nothing new since the fork: Git moves its name along the other branch
Merge commit	Both have moved: Git combines the changes in a commit with two parents
You control	The direction, and whether a merge commit may be made (--ff-only forbids it)
What comes out	A moved name, or a commit with two parents

Two kinds of merge

Fast-forward

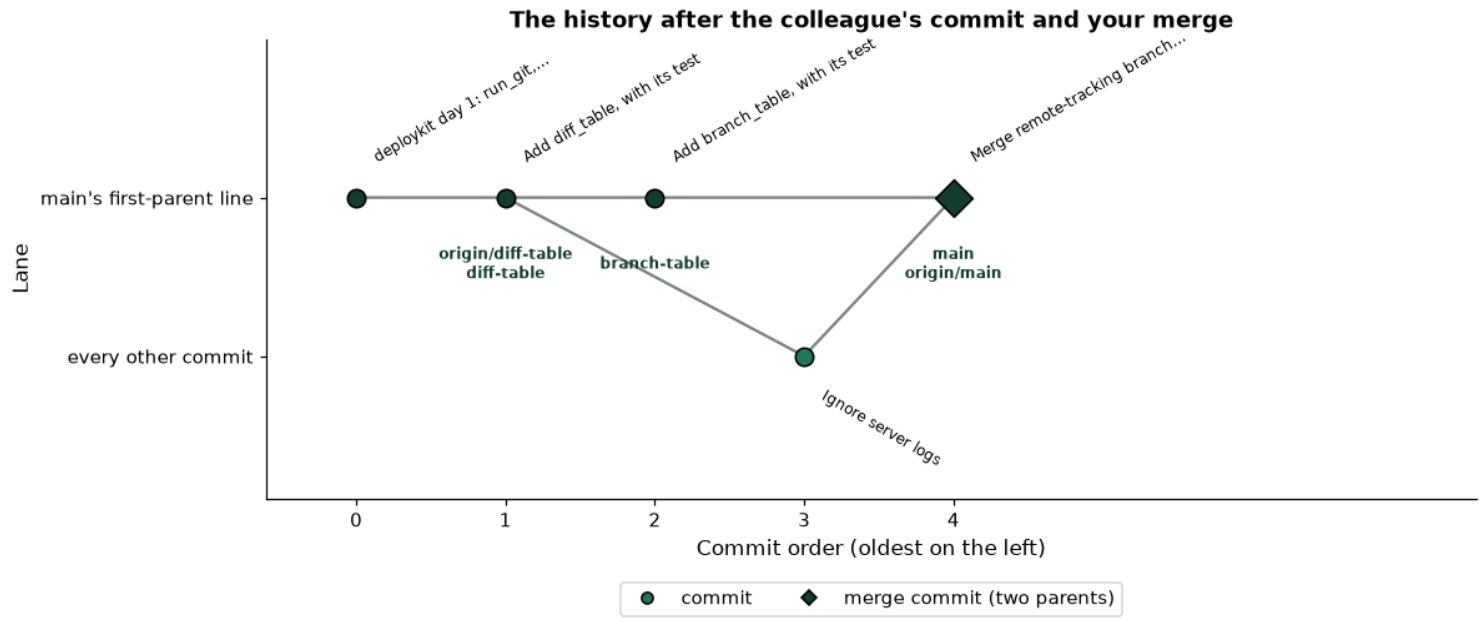
main has not moved: Git slides it forward. No new commit.

```
exit code: 1
To remote.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'remote.git'
```

`fetch`	[ahead 1, behind 1]
`pull --ff-only`	Not possible to fast-forward, aborting.
`merge origin/main`	Merge made by the 'ort' strategy.
`push`	accepted this time

```
*   Merge remote-tracking branch 'origin/main'
|\
| * Ignore server logs
* | Add branch_table, with its test
|/
* Add diff_table, with its test
```

The history, drawn



How to read it

Dot

one commit

Line

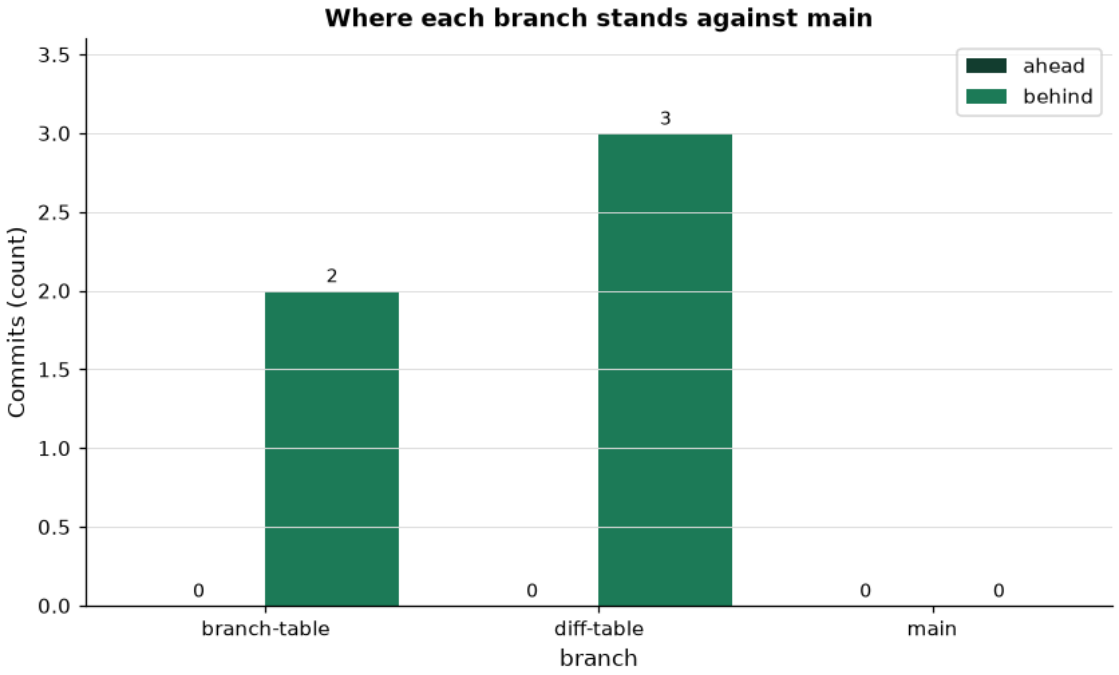
a commit to its parent

Diamond

a merge commit

The diamond is one commit with two parents. The top lane is main's first-parent line.

Where each branch stands



Branch	Ahead	Behind
branch-table	0	2
diff-table	0	3
main	0	0

Ahead is zero on every branch: everything on them is on main. A branch at zero ahead can be deleted.

How a conflict works

The problem	Both sides changed the same lines and Git cannot tell which version to keep
The method	Compare each side with the last shared commit; take a lone change, stop on a double one
The file	Both versions are written between three markers and the merge is left unfinished
You control	The text you keep, then git add and commit
What comes out	A merge commit like any other; a file still holding markers fails its tests

A conflict on purpose

```
deploykit.py
   188  <<<<<<< HEAD
   189  def conflict_files(folder) -> list[str]:
   197  =======
   198  def branch_table(folder):
   213  >>>>>>> main
```

Read both sides

HEAD

this branch: conflict_files

main

main's side: branch_table

both wanted

keep both, main's first

2

files in conflict

1

region in each

10

tests passed after

Git believes a file is resolved only when you stage it with git add.

How a pull request works

The problem	A change to a shared main needs a place to be read, checked, and discussed
The record	Not a Git object: GitHub's page for a base branch, a head branch, and the talk about them
Live	The diff is recomputed when either branch moves; checks and comments report to the page
You control	The two branches, the description, and the merge method
What comes out	A merge on main with the conversation attached

A pull request is a branch plus a conversation

What the platform printed, masked

```
Recorded 2026-10-12, the course's account, masked
> git push -u origin units-in-readme
branch 'units-in-readme' set up to track 'origin/units-in-readme'.
remote: Create a pull request for 'units-in-readme' on GitHub by visi
remote:      https://github.com/student/pfi-deploy-demo/pull/new/unit
To https://github.com/student/pfi-deploy-demo.git
 * [new branch]      units-in-readme -> units-in-readme
```

gh pr view, masked

state	**OPEN**
author	**student (student)**
additions	**1,473**
deletions	**1,465**

A README change of a few lines, with over a thousand additions: section 2.6 reads why.

How a review works

The problem	A description says what a change does; only the change shows what it does
The order	Cheapest read first: the diff stat, then the lines, then the changed words
The question	Does the size of the change match what the description says?
You control	How far down you read before you decide
What comes out	A comment tied to a line, and a decision

Review by diff

```
.gitignore  8
README.md  8
bondmath.py  1144
bondmath_excel_reference.csv  614
bondmath_excel_reference_tvm.csv  308
test_bondmath.py  856
```

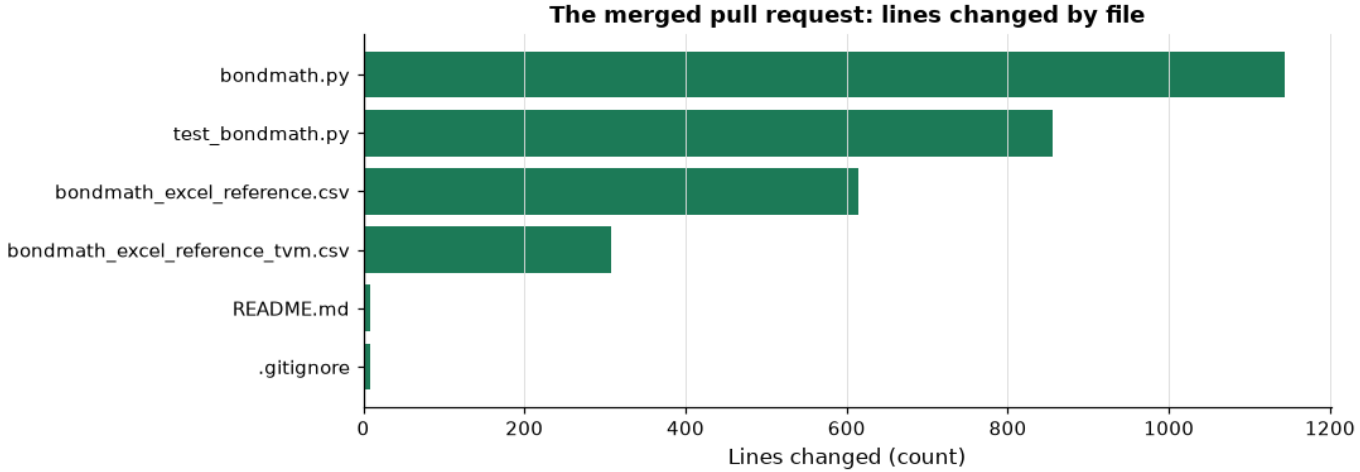
The course's own pull request

README: the convention change. Five files: only their line endings, from core.autocrlf switched off inside a clone. Read the file list before you merge.

```
def future_value(amount: float, [-rate_pct:-]{+rate:+} float, years: float, frequency: int = [-2)-]{
Units: amount in dollars (or per 100 of face), [-rate_pct-]{+rate+} in percent, years in years.
Excel: [-=FV(rate_pct/100/frequency,-]{+=FV(rate/100/frequency,+} frequency*years, 0, -amount)
```

```
branch: future_value(100, 5.0, 10) = 162.8895
main:   future_value(100, 5.0, 10) = 163.8616
```

What the merged pull request changed



File	Lines
.gitignore	8
README.md	8
bondmath.py	1,144
bondmath_excel_reference.csv	614
bondmath_excel_reference_tvm.csv	308
test_bondmath.py	856

One file changed on purpose. The other five changed only in their line endings. Read the file list before you merge.

How the three merge methods differ

The problem	One pull request, three ways to put it on main
Merge commit	Every commit of the branch, plus one commit that joins them
Squash	The whole change as one new commit; the branch's own commits stay behind
Rebase	The branch's commits replayed one by one on main: a straight line of copies
What comes out	The history every later reader and every deploy log will see

Three ways to merge

Merge commit

every commit, plus a merge commit

`merge --no-ff`

Squash

the branch's commits as one commit

`merge --squash`

Rebase

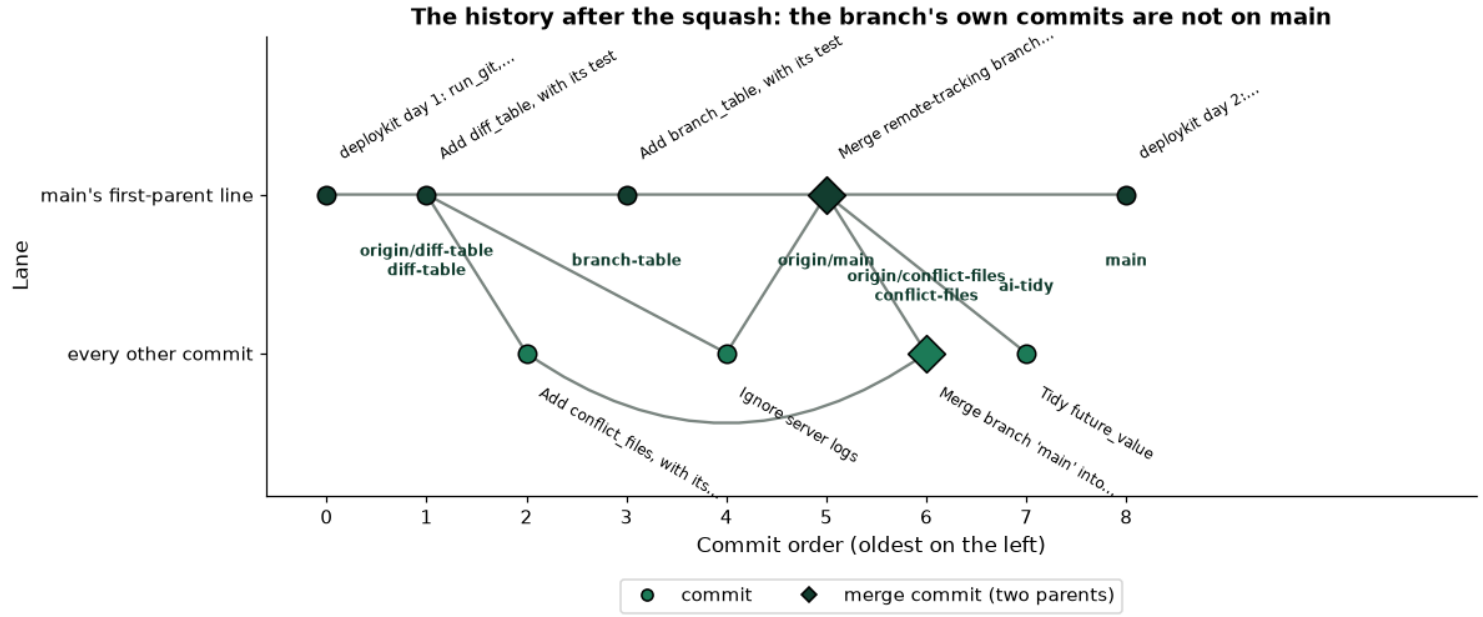
each commit copied, no merge commit

`commits replayed onto main`

```
Squash commit -- not updating HEAD
M  deploykit.py
M  test_deploykit.py
?? plotkit.py
```

The course squashes your own pull requests: one change, one commit on main.

The squash, drawn



How to read it

Top lane

main's first-parent line

Lower lane

every other commit

Diamond

a merge commit

A squash keeps one line; a merge commit keeps both. Main holds the sum, the branch holds the history.

Deleting merged branches

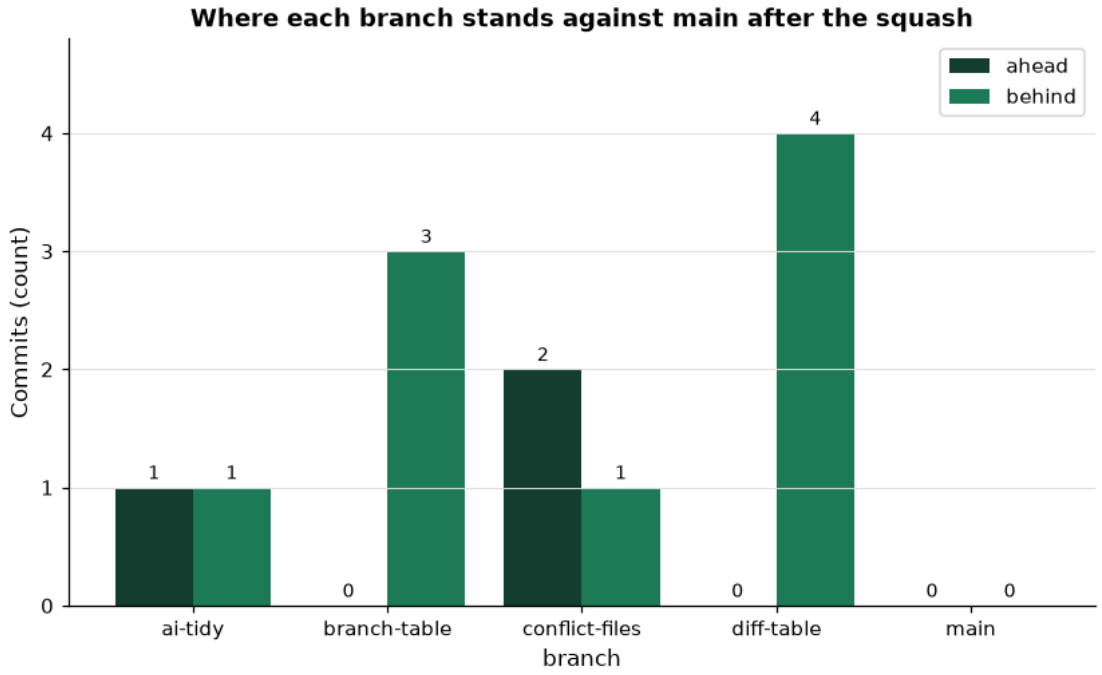
```
ai-tidy         ahead  1  behind  1
branch-table    ahead  0  behind  3
conflict-files  ahead  2  behind  1
diff-table      ahead  0  behind  4
main            ahead  0  behind  0
```

1. `git push origin --delete <branch>`
2. `git branch -d <merged branch>`
3. `git branch -D`, only when you know why `-d` refused

```
error: the branch 'conflict-files' is not fully merged
```

A squash makes a new commit: the branch's own commits never reach main, so -d refuses.

After the squash



Branch	Ahead	Behind
ai-tidy	1	1
branch-table	0	3
conflict-files	2	1
diff-table	0	4
main	0	0

conflict-files is 2 ahead though every line of it is on main: a squash makes a new commit.

Excel and Git

Excel side by side: none today

Excel has no branch, no merge, and no conflict to resolve: two copies stay two workbooks.

```
To remote.git
 - [deleted]         conflict-files
 - [deleted]         diff-table
```

The day's module commit: a squash merge of conflict-files. 10 tests on main.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: git checkout --theirs on every conflicted file: the reviewed side is dropped, and the merge still commits.

Code that runs is not the same as code that is right.

What you can do now

- main is what is deployed; work on a branch
- Read a change with diff_table before it merges
- Fast-forward, merge commit, a rejected push
- Resolve a conflict by reading both sides
- Pull requests, squash, and deleting branches

Week 1, Day 3 CI with GitHub Actions